

Dossier Personnel

BTS CIEL SESSION 2026 - IR

E6 – PROJET TECHNIQUE

Pilotage d'éclairage scénique multi-univers Art-Net/DMX



Lycée : Touchard-Washington		Ville : LE MANS	
Nom du projet :	Pilotage d'éclairage scénique multi-univers Art-Net/DMX		
N° du projet :	TW3		
Projet nouveau	Oui ☒ Non ☐	Projet interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants :	4 étudiants
Professeurs responsables	François MARTIN , Philippe CRUCHET, Didier BERNARD, Anthony LE CREN, Jilali KHAMLACH, François QUEREC		

Sommaire

Table des matières

1. Cas d'utilisation.....	3
1.1. Obtenir la liste des scènes.....	3
1.2. Lancer une scène.....	4
1.3. Tester un équipement DMX.....	5
2. Diagrammes de séquences.....	6
2.1. Obtenir la liste des scènes.....	6
2.2. Lancer une scène.....	7
2.3. Tester un équipement DMX.....	8
3. Protocoles de communication.....	9
3.1. TCP/IP.....	9
3.2. Art-Net.....	10
3.3. DMX512.....	11
4. Outils.....	13
4.1. Qt Creator.....	13
4.1.1. Architecture logicielle :.....	15
4.3. Github.....	17
5. Diagrammes de classe.....	18
6. Fiches de test.....	19
7. Diagramme de Gantt.....	21
7.1. Diagramme de Gantt pour la tablette.....	22
8. Compétences acquises.....	23
9. Perspectives d'améliorations.....	24

1. Cas d'utilisation

1.1. Obtenir la liste des scènes

Nom du cas d'utilisation	Obtenir la liste des scènes
<u>Pré-conditions</u>	La tablette est allumé et l'application est lancé
<u>Post-conditions</u>	Un indicatif montre que la tablette est connecté et a donc reçu toute les info de la BDD
<u>Scénario principal</u>	- L'opérateur arrive au niveau du menu pour s'identifier. - En arrière plan la tablette essaye de se connecte à la BDD. - Quand la connexion est établie, la tablette va chercher toute les info de la BDD (dont la liste des scènes)
<u>Alternatives</u>	- L'opérateur arrive au niveau du menu pour s'identifier. - En arrière plan la tablette essaye de se connecte à la BDD. - La tablette n'arrive pas à se connecté et retente en boucle.

1.2. Lancer une scène

Nom du cas d'utilisation	Lancer une scène
<u>Pré-conditions</u>	• L'application est lancé • La tablette est connecté à la Raspberry • L'opérateur est identifier
<u>Post-conditions</u>	• La trame JSON pour lancé la scène est envoyé • La scène s'exécute instantanément
<u>Scénario principal</u>	1. L'opérateur ouvre le menu scène 2. Il appui sur le bouton lancer d'une scène 3. La trame JSON pour lancé la scène est envoyé à la Raspberry
<u>Alternatives</u>	• Arrivé au menu scène, aucune scène ne s'affiche (tablette déconnecté ou aucune scène dans la BDD) • L'opérateur appui sur le bouton lancer mais aucune trame ne s'envoie

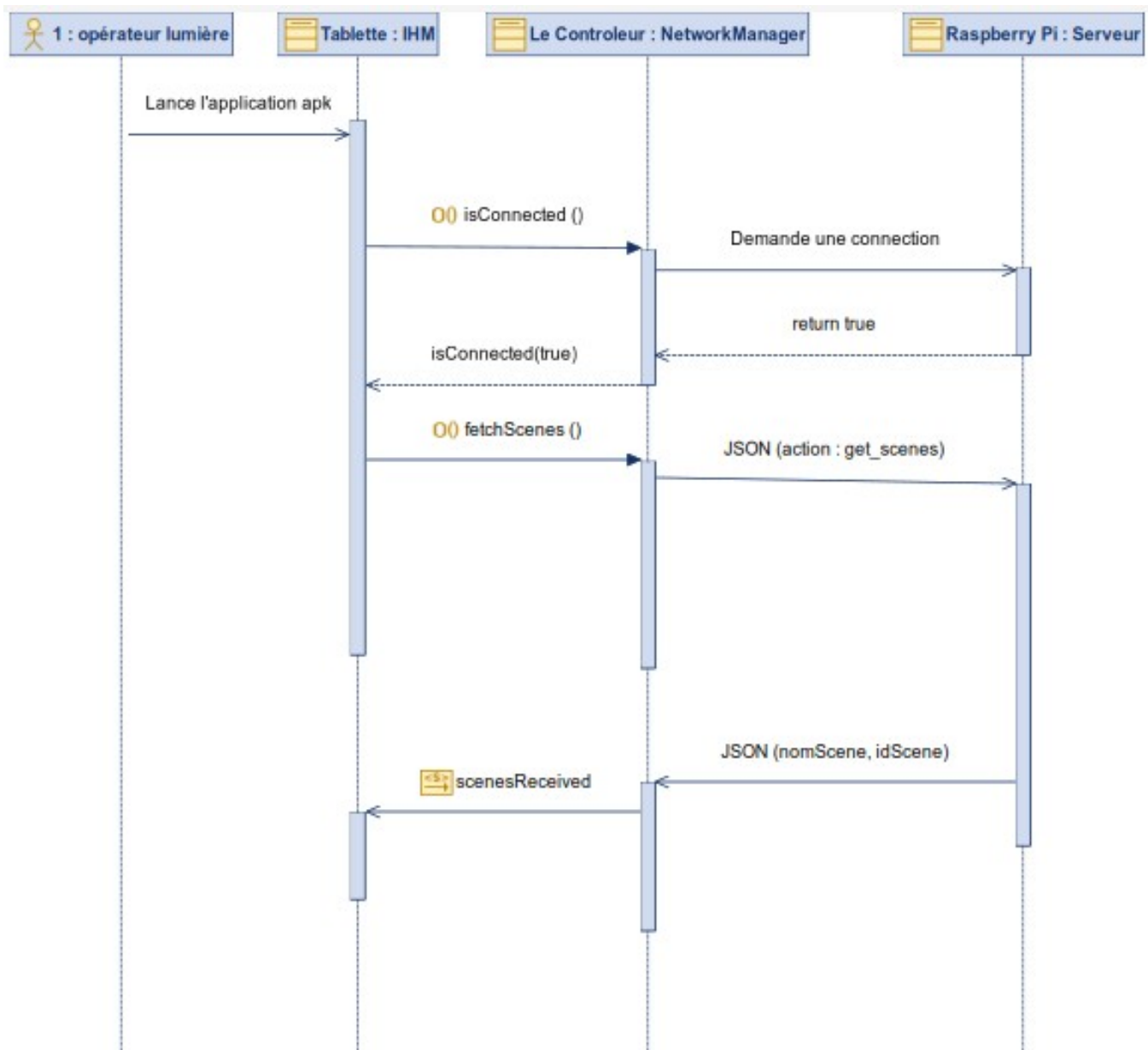
1.3. Tester un équipement DMX

Nom du cas d'utilisation	Tester un équipement DMX
<u>Pré-conditions</u>	• L'application est lancée • La tablette est connectée à la Raspberry • L'opérateur est identifié
<u>Post-conditions</u>	• La trame pour modifier la valeur d'un canaux est envoyée • L'équipement DMX réagit en fonction des changements demandés
<u>Scénario principal</u>	1. L'opérateur ouvre le menu Contrôle 2. Il choisit l'univers 3. Puis il fait glisser un ou plusieurs sliders pour modifier les valeurs des canaux DMX des projecteurs 4. Les trames s'envoient pour chaque valeur changer
<u>Alternatives</u>	• Arrivé au menu Contrôle, aucun univers ni curseur ne s'affiche (tablette déconnectée ou aucun univers ou équipement DMX dans la BDD)

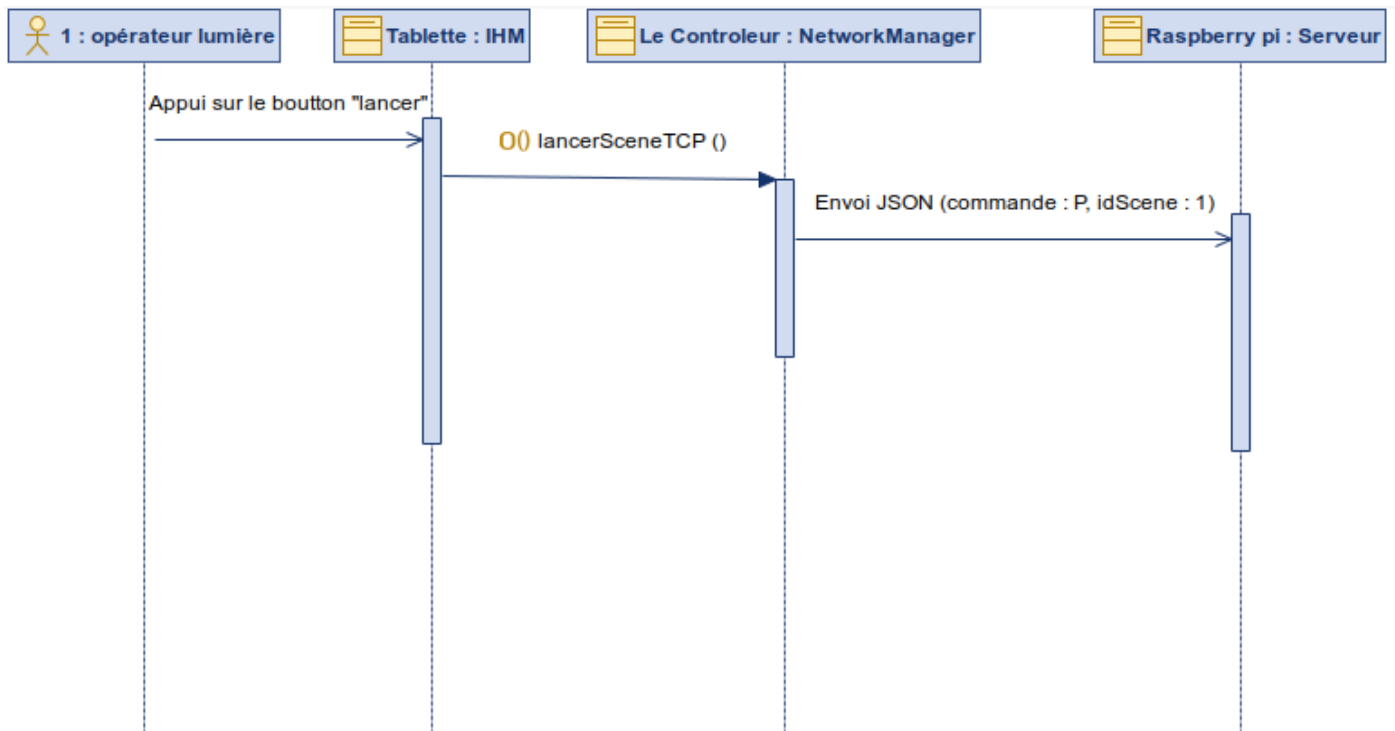
2. Diagrammes de séquences

Après avoir présenté l'ensemble de mes cas d'utilisation et les outils que j'ai choisi, il est désormais possible de représenter des diagrammes de séquences complets, représentant le vrai programme.

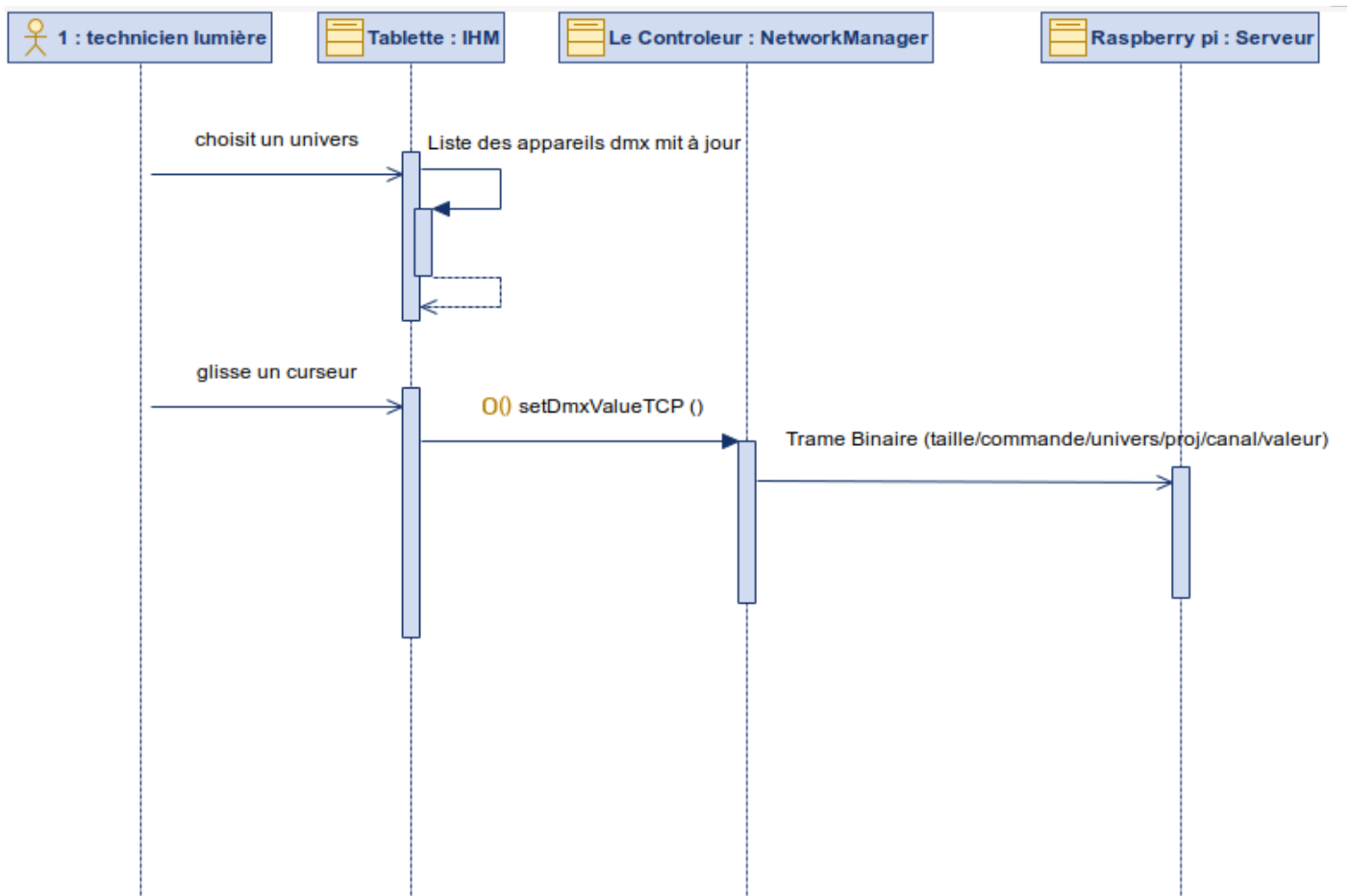
2.1. Obtenir la liste des scènes



2.2. Lancer une scène



2.3. Tester un équipement DMX



3. Protocoles de communication

3.1. TCP/IP

TCP/IP est la suite de protocoles fondamentale d'Internet et des réseaux locaux. Elle définit comment les données sont découpées, adressées, transmises et réassemblées entre deux machines.

TCP (couche transport) garantit que chaque paquet envoyé arrive à destination, dans le bon ordre et sans erreur. Si un paquet est perdu, il est automatiquement renvoyé. C'est ce qui le distingue d'UDP, qui lui n'offre aucune garantie de livraison.

IP (couche réseau) s'occupe de l'adressage et du routage : il identifie la source et la destination grâce aux adresses IP (ex. : 192.168.1.10) et achemine les paquets sur le réseau.

Utilisation dans le projet

Dans le DMX Equipment Manager, TCP est utilisé pour envoyer les commandes de lancement de scènes depuis le PC vers le Raspberry Pi. La connexion est établie via la classe QTcpSocket de Qt.

Exemple de commande envoyée lors du lancement d'une scène :

```
QJsonObject obj;  
obj["commande"] = "P";  
obj["idScene"] = 13;  
QJsonDocument doc(obj);  
QByteArray data = doc.toJson(QJsonDocument::Compact);  
socketClient.write(data);
```

La trame JSON envoyée sur le réseau ressemble à ceci :

```
{ "commande": "P", "idScene": 13 }
```

Le Raspberry Pi reçoit cette trame, extrait l'identifiant de la scène, récupère les valeurs DMX associées en base de données et les transmet aux équipements via Art-Net.

3.2. Art-Net

Art-Net est un protocole réseau développé par Artistic Licence en 1998. Il permet de transporter le signal DMX 512 sur un réseau Ethernet standard via UDP (User Datagram Protocol). Contrairement à TCP, UDP n'établit pas de connexion et ne garantit pas la livraison — ce qui est acceptable en éclairage scénique

Avantage principal : Art-Net permet de transmettre jusqu'à 32 768 univers DMX sur un seul réseau IP, là où le DMX classique se limite à 512 canaux par câble physique.

Un paquet Art-Net (OpDmx) se compose d'un en-tête fixe suivi des 512 valeurs de canaux DMX. Voici sa structure détaillée :

Octet(s)	Champ	Valeur	Description
0 — 7	ID	41 72 74 2D 4E 65 74 00	Signature ASCII "Art-Net" + '0'
8 — 9	OpCode	00 50	OpDmx : commande d'envoi DMX (little-endian)
10 — 11	ProtVer	00 0E	Version du protocole Art-Net (14)
12	Sequence	00	Séquence (0 = désactivée)
13	Physical	00	Port physique source
14 — 15	Universe	00 00	Numéro d'univers DMX (0 à 32767)
16 — 17	Length	02 00	Nombre de canaux DMX (512 = 0x0200)
18 — 529	Data	00 FF 00 80 ...	Valeurs des 512 canaux DMX (0 à 255)

Utilisation dans le projet

Dans le DMX Equipment Manager, chaque univers DMX est associé à une adresse IP correspondant au nœud Art-Net (boîtier convertisseur Ethernet/DMX ou Raspberry Pi). L'adresse IP et le numéro d'univers sont configurés par l'utilisateur et stockés en base de données.

Lorsqu'une scène est lancée, le Raspberry Pi récupère les valeurs des canaux et les envoie au nœud Art-Net correspondant sous forme de paquets UDP sur le port 6454 (port standard Art-Net).

3.3. DMX512

DMX 512 (Digital Multiplex with 512 pieces of information) est un protocole de communication numérique standardisé (ANSI E1.11) utilisé pour le contrôle des équipements d'éclairage scénique depuis 1986. Il permet de contrôler jusqu'à 512 canaux indépendants sur un seul câble, chaque canal pouvant prendre une valeur entre 0 et 255.

Chaque appareil DMX occupe un certain nombre de canaux consécutifs à partir de son adresse de départ. Par exemple, un projecteur à l'adresse 1 avec 4 canaux occupe les canaux 1, 2, 3 et 4.

Le signal DMX est transmis en série à 250 kbauds via RS-485. Une trame DMX complète se compose des éléments suivants :

Élément	Durée	Description
Break	$\geq 88 \mu s$	Signal bas indiquant le début d'une nouvelle trame
Mark After Break (MAB)	$\geq 8 \mu s$	Signal haut séparant le Break du premier octet
Start Code	1 octet (44 μs)	Code de démarrage, toujours 0x00 pour DMX standard
Slot 1 à 512	1 octet chacun	Valeur de chaque canal DMX (0 à 255)
Mark Time Between Frames	Variable	Temps entre deux trames (optionnel)

Voici une représentation simplifiée d'une trame DMX pour 4 canaux :

BREAK	→	_____
MAB	→	-----
Start Code	→	0x00 (DMX standard)
Canal 1	→	0xFF (255 – pleine intensité)
Canal 2	→	0x80 (128 – mi-intensité)
Canal 3	→	0x1A (26 – Rouge : 10%)
Canal 4	→	0x00 (0 – éteint)
...	→	0x00 (canaux 5 à 512 à 0)

Dans le DMX Equipment Manager, chaque équipement est enregistré avec son adresse DMX de départ et son nombre de canaux. L'application calcule automatiquement le canal absolu de chaque canal :

$$\text{Canal absolu} = \text{adresseDepart} + \text{numeroCanal} - 1$$

Exemple : Projecteur à l'adresse 5, canal 3
 → Canal absolu = 5 + 3 - 1 = 7

Voici un exemple d'adressage pour trois équipements sur un même univers :

Équipement	Adresse de départ	Nb canaux	Canaux occupés
Projecteur Contest 1	1	4	1, 2, 3, 4
Ruban LED	5	6	5, 6, 7, 8, 9, 10
Lyre iRock 7	11	7	11, 12, 13, 14, 15, 16, 17

Deux équipements ne doivent jamais occuper les mêmes canaux DMX sur un même univers, sous peine de conflit de commandes. Le DMX Equipment Manager vérifie cette règle à l'enregistrement et affiche un message d'erreur si une plage de canaux se chevauche avec un équipement déjà enregistré. Il néanmoins possible de mettre deux équipements sur les mêmes canaux afin de les piloter en même temps.

Chaque univers peut accueillir au maximum 512 canaux. Si la somme des canaux de tous les équipements dépasse 512, les appareils au-delà de cette limite ne répondront pas aux commandes.

4. Outils

4.1. Qt Creator

Qt est un framework C++ multiplateforme publié sous licence LGPL par The Qt Company. Il couvre l'intégralité du cycle d'une application : interface graphique dynamique (QML / Qt Quick pour le tactile), réseau (QtNetwork), traitement de données (JSON, binaire), threads, timers et bien plus — le tout avec une seule API cohérente sur Windows, Linux, Android et macOS.

Son mécanisme central est le système Signaux/Slots : les objets communiquent entre eux sans couplage direct, ce qui simplifie la gestion des événements asynchrones (connexion TCP, réception des accusés de réception, interactions tactiles de l'opérateur).

Dans ce projet, Qt est déployé pour développer l'application de la Tablette Android (interface graphique fluide en QML couplée au backend C++ NetworkManager) qui communique en direct avec le Raspberry Pi (serveur DMX).

QDebug — Sortie console

Trace les événements réseau et les erreurs SQL. Aucun setup, simple include.

```
QDebug() << "Envoi JSON :" << data;
```

```
QDebug() << "Erreur insertion CANAUX :" << query.lastError().text();
```

QTcpSocket — Communication réseau

Connexion TCP PC ↔ Raspberry Pi, envoi de commandes JSON.

```
socketClient.connectToHost(ip, port);
```

```
socketClient.write(doc.toJson(QJsonDocument::Compact));
```

QJsonObject / QJsonDocument — JSON

Sérialisation des commandes et parsing des fichiers de config équipements.

```
QJsonObject obj;
```

```
obj["commande"] = "P"; obj["idScene"] = idScene;
```

```
QByteArray data = QJsonDocument(obj).toJson(QJsonDocument::Compact);
```

QDataStream & QBuffer — Flux Binaire Temps Réel

Préparation et formatage des trames binaires ultra-compactes (9 octets) pour le contrôle DMX instantané via les curseurs de la tablette, sans saturer le réseau (approche "Fire and Forget").

```
// Préparation du tampon mémoire
```

```
QBuffer tampon;
```

```
tampon.open(QIODevice::WriteOnly);
```

```
QDataStream out(&tampon);
```

```
// Construction de la trame binaire avec les types stricts
```

```
char commande = 'C';
```

```
out << (quint16)0 << commande << (quint8)univers <<  
(quint16)projecteur << (quint16)canal << (quint8)valeur;
```

4.1.1. Architecture logicielle :

L'application de la tablette repose sur une séparation stricte entre le moteur de l'application (backend) et l'interface utilisateur (frontend). Qt permet de combiner trois langages pour optimiser à la fois les performances et la fluidité visuelle :

C++ — Le cœur réseau et la logique système (Backend)

Le C++ est utilisé pour toute la partie "invisible" de l'application. Langage compilé et très performant, il est chargé d'établir les connexions TCP, de formater les trames binaires, de parser les JSON complexes et de gérer la mémoire. C'est la classe **NetworkManager** qui effectue ce travail lourd en tâche de fond pour garantir le contrôle DMX en temps réel sans jamais figer l'écran.

```
// Le C++ gère le réseau et signale l'interface quand c'est prêt

void NetworkManager::onTcpReadyRead() {

    // ... traitement lourd des paquets ...

    emit scenesReceived(arr.toVariantList()); // Signal envoyé à l'interface
}
```

QML (Qt Modeling Language) — L'Interface Utilisateur tactile (Frontend)

Le QML est un langage déclaratif (basé sur une syntaxe proche du JSON) conçu spécifiquement par Qt pour la création d'interfaces graphiques fluides, animées et adaptées aux écrans tactiles. Il bénéficie de l'accélération matérielle graphique. Le QML se contente d'afficher les éléments (Curseurs, Boutons, Listes) et d'écouter les signaux émis par le C++ via des connexions (Connections) pour se mettre à jour instantanément.

```
// Le QML affiche l'interface et écoute le backend C++

Slider {

    id: mySlider

    from: 0; to: 255
```

```
// Émission de la valeur vers le C++ lors d'un glissement  
onValueChanged: myNetwork.setDmxValueTCP(univers, projecteur,  
canal, value)  
}
```

JavaScript (JS) — La logique d'interface (La glue entre QML et C++)

Le JavaScript est embarqué de manière native à l'intérieur du QML. Il est utilisé pour traiter la petite "logique métier" directement liée à l'interface, sans avoir besoin de solliciter le C++. Par exemple, le JavaScript est utilisé pour calculer la fonctionnalité du "Blackout Général", boucler sur la mémoire locale des curseurs (`savedSliderValues`), ou arrondir des valeurs mathématiques avant leur envoi.

```
// Fonction JavaScript (dmxLogic.js) appelée depuis le QML  
function triggerGlobalBlackout(savedValues, universes, myNetwork) {  
    var temp = Object.assign({}, savedValues);  
    // ... boucle sur tous les projecteurs pour forcer la valeur à 0 ...  
    return temp;  
}
```


4.3. Github

GitHub est utilisé comme plateforme de gestion de version et de collaboration. Cet outil s'est révélé particulièrement utile pour organiser, suivre et sécuriser le développement du code tout au long du projet. L'utilisation de GitHub présente plusieurs avantages concrets :

- Historique des modifications : chaque changement apporté au code source est enregistré, ce qui permet de revenir à une version antérieure en cas d'erreur ou de dysfonctionnement.
- Travail collaboratif : GitHub facilite le travail en équipe grâce au partage de dépôts, à la gestion des branches, et à l'intégration de contributions multiples sans conflit. Sauvegarde sécurisée : le code est stocké en ligne, évitant toute perte liée à une défaillance locale.
- Professionnalisation : GitHub est un outil couramment utilisé dans le milieu professionnel. Son utilisation dans le cadre de ce projet permet d'acquérir de bonnes pratiques de développement logiciel.

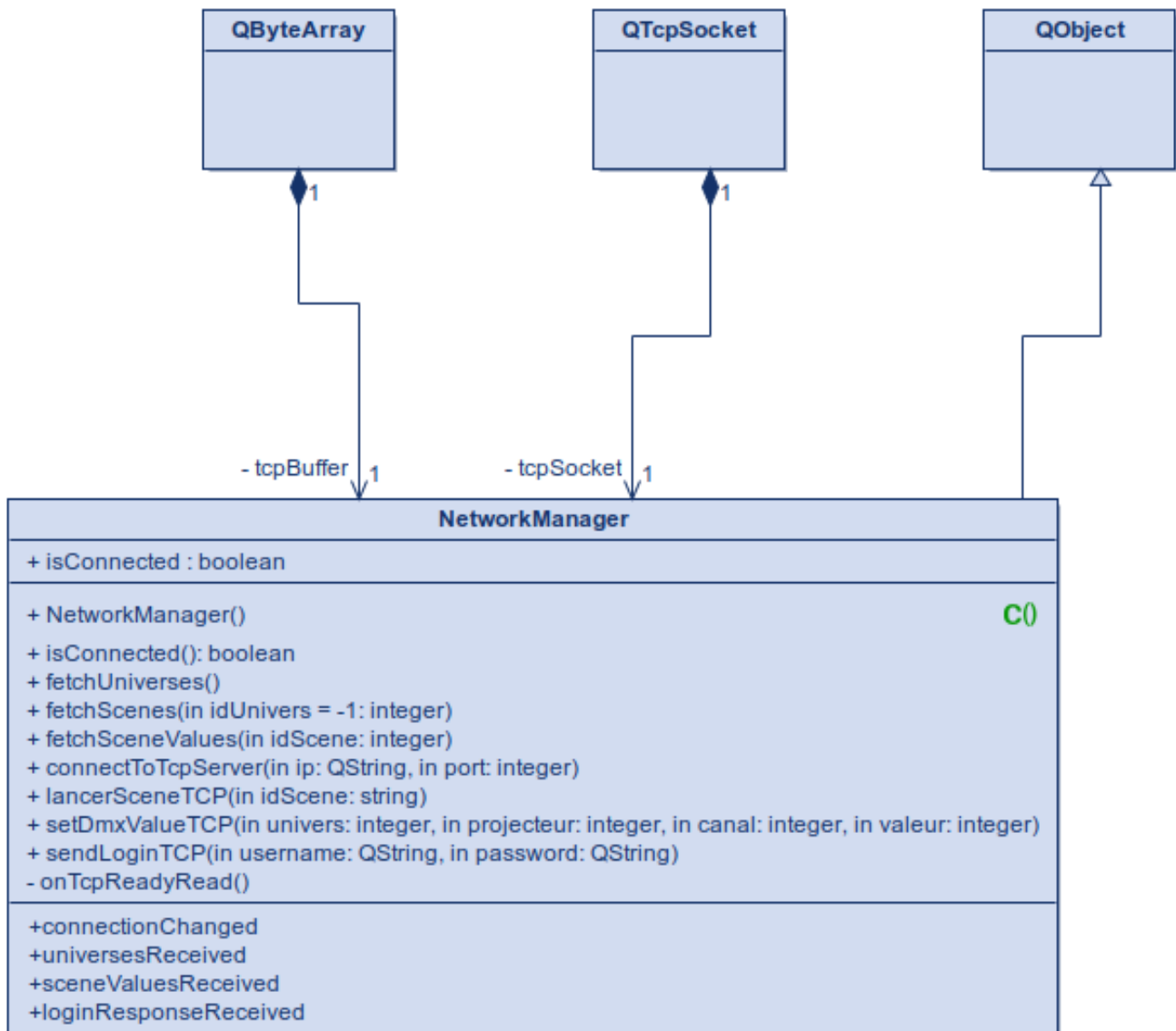
Grâce à GitHub, la gestion du code a été plus fluide, plus structurée et mieux adaptée à la collaboration entre les membres du projet. Cela a contribué à la réussite technique et organisationnelle du pilotage de l'éclairage scénique en Art-Net et DMX512.

Le lien pour accéder à notre projet final et nos dossiers (commun et personnels) se trouve ici :

https://github.com/aallard12/project_DMX_Art-Net



5. Diagrammes de classe

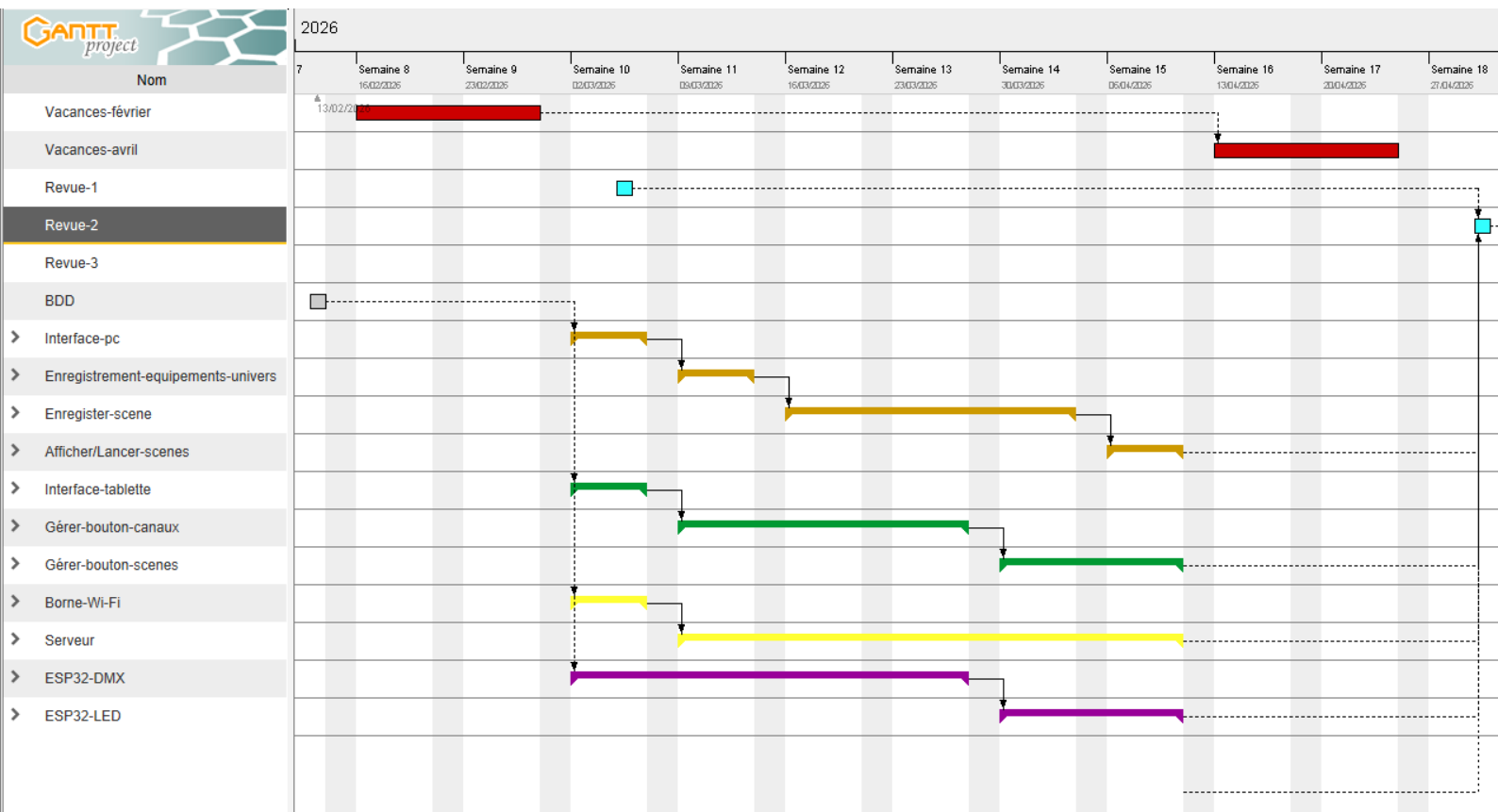


6. Fiches de test

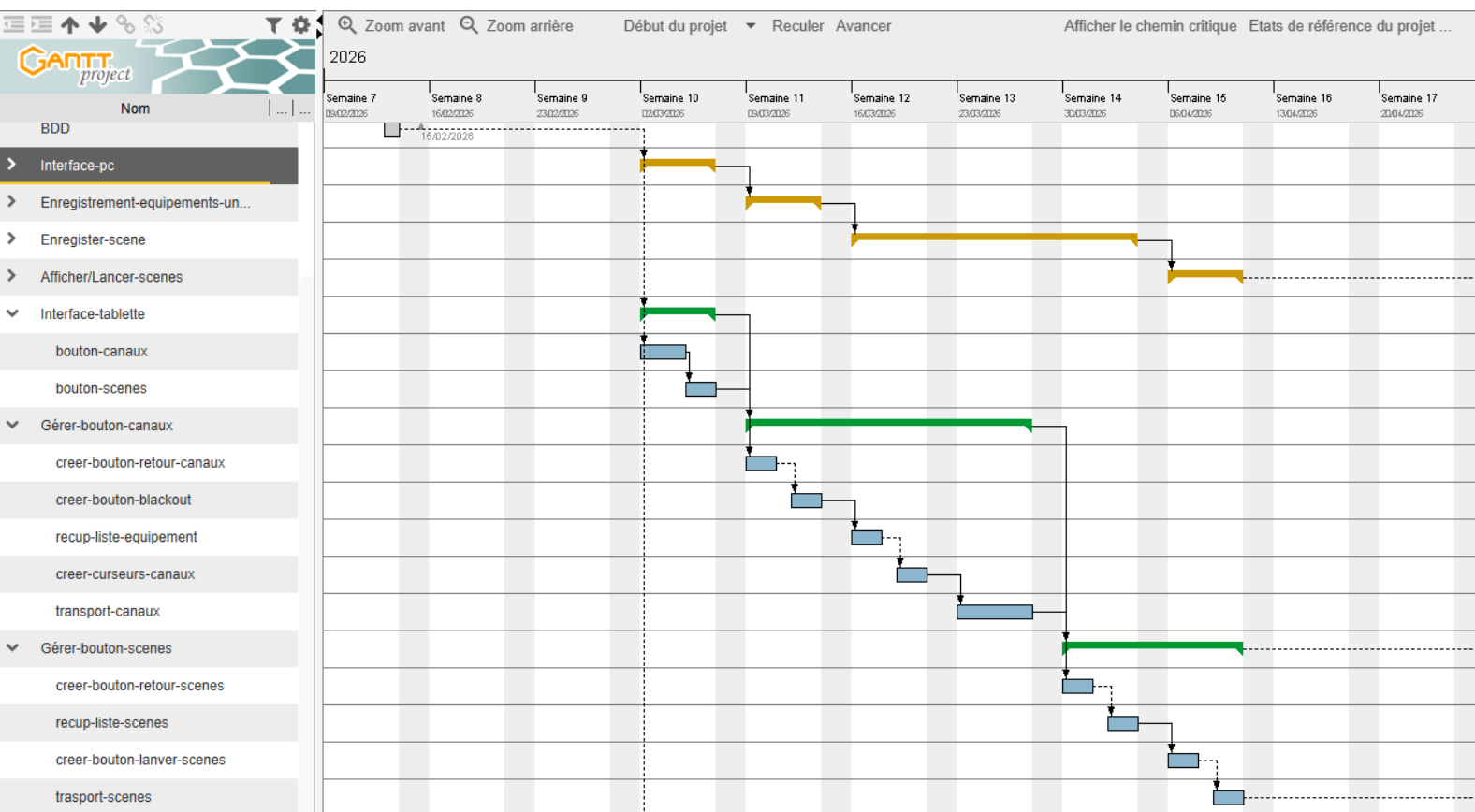
Référence du module testé : La classe NetworkManager		F1.1																												
Date du test :	Jeudi 30 avril 2026																													
Type du test :	Test fonctionnel																													
Objectif du test :	Valider la capacité de l'application à établir une connexion TCP avec le serveur DMX (Raspberry Pi), à maintenir un état de connexion stable, et à récupérer correctement les données de la base de données (Univers et Scènes).																													
<u>Conditions du test</u>																														
État initial Le serveur de base de données MariaDB est actif sur le serveur distant (192.168.1.20). Les tables UNIVERS et SCENES contiennent les données suivantes : <table border="1"> <thead> <tr> <th colspan="2">Table : UNIVERS</th> </tr> <tr> <th><u>idUnivers</u></th> <th>adresselp</th> </tr> </thead> <tbody> <tr> <td>1</td> <td>192.168.1.31</td> </tr> <tr> <td>2</td> <td>192.168.1.32</td> </tr> <tr> <td>3</td> <td>192.168.1.33</td> </tr> </tbody> </table> <table border="1"> <thead> <tr> <th colspan="2">Table : SCENES</th> </tr> <tr> <th><u>idScene</u></th> <th>nomScene</th> </tr> </thead> <tbody> <tr> <td>1</td> <td><u>RougeUniv1</u></td> </tr> <tr> <td>2</td> <td>VertUniv2</td> </tr> <tr> <td>3</td> <td><u>BleuUniv3</u></td> </tr> <tr> <td>4</td> <td><u>EffetUniv3</u></td> </tr> <tr> <td>5</td> <td>BLACKOUT Univers 1</td> </tr> <tr> <td>6</td> <td>BLACKOUT Univers 2</td> </tr> <tr> <td>7</td> <td>BLACKOUT Univers 3</td> </tr> </tbody> </table>		Table : UNIVERS		<u>idUnivers</u>	adresselp	1	192.168.1.31	2	192.168.1.32	3	192.168.1.33	Table : SCENES		<u>idScene</u>	nomScene	1	<u>RougeUniv1</u>	2	VertUniv2	3	<u>BleuUniv3</u>	4	<u>EffetUniv3</u>	5	BLACKOUT Univers 1	6	BLACKOUT Univers 2	7	BLACKOUT Univers 3	Environnement du test • Tablette avec l'application développée avec QT 6.8 installé • Tablette connecté au routeur en wifi • Raspberry lancé et BDD opérationnel
Table : UNIVERS																														
<u>idUnivers</u>	adresselp																													
1	192.168.1.31																													
2	192.168.1.32																													
3	192.168.1.33																													
Table : SCENES																														
<u>idScene</u>	nomScene																													
1	<u>RougeUniv1</u>																													
2	VertUniv2																													
3	<u>BleuUniv3</u>																													
4	<u>EffetUniv3</u>																													
5	BLACKOUT Univers 1																													
6	BLACKOUT Univers 2																													
7	BLACKOUT Univers 3																													

<u>Procédures du test</u>		
N°	Opérations	Résultats attendus
1	Lancer l'application DMX sur la tablette alors que le serveur est allumé.	- L'indicateur de connexion passe au VERT ("CONNECTÉ"). - Dans le moniteur de diagnostic : Log SYS: Connecté. - La liste des univers est chargée automatiquement.
2	Ouvrir la page "Scènes" et cliquer sur "Rafraîchir".	- Log SEND: GET Scènes dans le diagnostic. - Apparition des scènes. - Log RECV: [{"idScene": 1, "nomScene": "Strob"}, ...].
3	Couper le Wi-Fi de la tablette pendant 10 secondes puis le réactiver.	- L'indicateur passe au ROUGE ("DÉCONNECTÉ"). - Le Timer tente une reconnexion toutes les 5s. - Au retour du Wi-Fi, l'état repasse à "CONNECTÉ" automatiquement.
4	Envoyer une valeur DMX via un curseur dans "Contrôle".	- Log SEND: DMX U:1 P:1 C:1 V:255. - La trame binaire est envoyée sans erreur sur la socket.

7. Diagramme de Gantt



7.1. Diagramme de Gantt pour la tablette



8. Compétences acquises

Sur le plan technique :

- J'ai vraiment progressé en QML et C++ avec Qt, surtout sur la partie séparation entre l'interface (frontend) et le réseau (backend) via l'utilisation stricte des signaux, slots et macros Q_INVOKABLE.
- La communication TCP m'a demandé du temps à comprendre, mais concevoir un protocole hybride (JSON pour les requêtes structurées, trames binaires 9 octets pour le temps réel) a été un vrai apprentissage.
- Le déploiement logiciel croisé m'a montré comment configurer un environnement Linux pour compiler une application, générer un livrable Android (.apk) et le déployer sur tablette via USB.

Sur le plan professionnel :

- J'ai compris l'importance du contexte métier : avant de coder, il a fallu définir la vraie valeur ajoutée de la tablette (permettre à l'opérateur de se déplacer sur scène) par rapport à la régie fixe sur PC.
- Prendre le temps de concevoir des diagrammes UML stricts (différence entre agrégation et composition, variables génériques, visibilité des attributs) m'a évité beaucoup de problèmes de conception en amont.
- La gestion de projet m'a appris à analyser objectivement les écarts entre le diagramme de Gantt prévisionnel et le déroulé réel, et à confronter mes réalisations au cahier des charges initial.
- Faire face à de vrais défis techniques (récupération asynchrone des données, complexité du lien C++/JavaScript) m'a appris à chercher méthodiquement plutôt que de modifier du code au hasard.

En conclusion, ce projet m'a beaucoup apporté, autant techniquement qu'humainement. Je repars avec des compétences concrètes que je n'avais pas en début d'année, et surtout avec une méthode de travail que je compte bien garder pour la suite.

9. Perspectives d'améliorations

- **Paramétrage réseau dynamique** : Aujourd'hui, l'adresse IP et le port du serveur sont codés en dur dans l'application. Prévoir une page de configuration au démarrage permettrait de s'adapter facilement à n'importe quel réseau de spectacle sans avoir à recompiler l'APK.
- **Modification et sauvegarde de scènes** : L'envoi de commandes DMX fonctionne très bien. La suite logique serait de permettre à l'opérateur, après avoir ajusté une ambiance lumineuse directement depuis la scène physique, de sauvegarder ou d'écraser cette scène dans la base de données depuis la tablette.
- **Ajouter une création de compte** : Pouvoir au menu connexion créer un profil avec nom, mot de passe, adresse mail directement sur la tablette. Au lieu que les créations se fassent uniquement sur le PC.
- **Sécurisation** : Pour l'instant, le mot de passe est seulement chiffré dans la BDD mais nous avons oublié un détail. Au lieu d'envoyer un mot de passe en clair vers le serveur pour se connecter, il faudrait le hacher pour rendre la sécurité opérationnelle.

Fonctionnalité phare :

- Plan de scène 2D interactif : l'affichage des projecteurs sous forme de liste fonctionne bien, mais la majorité des régisseurs préfèrent une approche spatiale. Intégrer un plan interactif où l'opérateur pourrait placer ses équipements, puis tapoter directement sur un projecteur graphique pour faire apparaître ses curseurs, exploiterait à 100% l'aspect tactile et mobile de la tablette.

En conclusion, l'application est aujourd'hui fonctionnelle et couvre l'ensemble des besoins du projet. Ces améliorations ne remettent pas en cause ce qui a été réalisé mais elles viendraient compléter et solidifier le projet pour une utilisation professionnelle sur le long terme.